Mini Project Assignment: Feature-Based Traffic Monitoring System

ISHIKA

2301010303

BTECH CSE(E) CORE

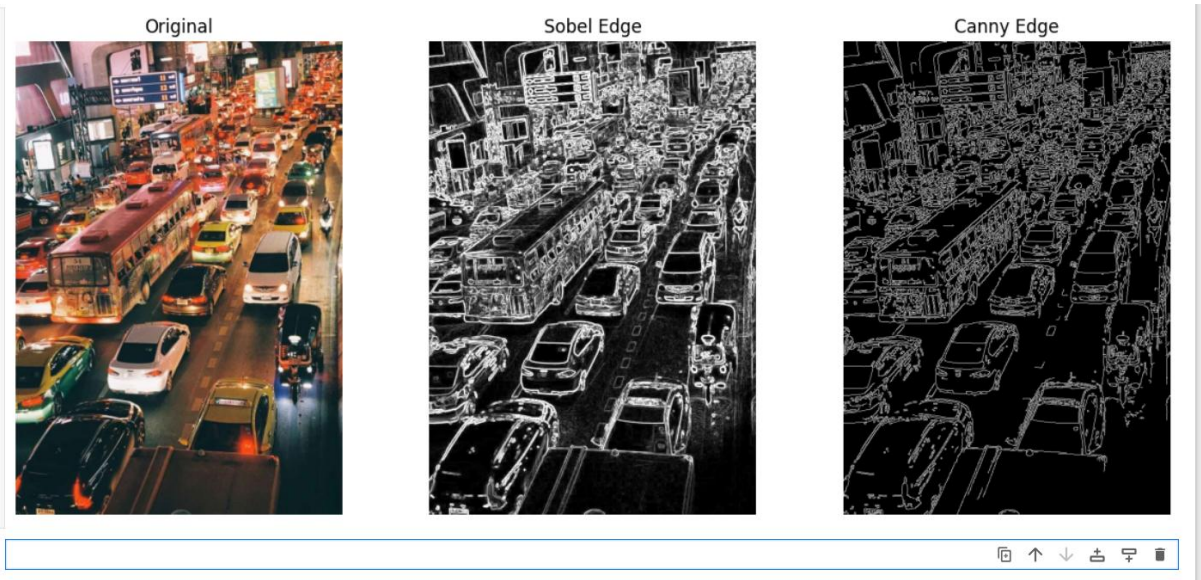
Assignment Tasks

Task 1: Edge Detection

- Apply Sobel operator
- Apply Canny edge detector
- Compare edge quality

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Read image
img = cv2.imread("img.jpg")

if img is None:
    print("Error: Image not found")
else:
    # Convert to RGB for display
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Convert to grayscale
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # ------------------------------
    # Sobel Edge Detection
    # ------------------------------
    sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
    sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

    sobel = cv2.magnitude(sobel_x, sobel_y)
    sobel = cv2.convertScaleAbs(sobel)

    # ------------------------------
    # Canny Edge Detection
```

```
# ----------------------------
canny = cv2.Canny(gray, 100, 200)

# ----------------------------
# Display Results
# ----------------------------
plt.figure(figsize=(12,5))

plt.subplot(1,3,1)
plt.imshow(img_rgb)
plt.title("Original")
plt.axis('off')

plt.subplot(1,3,2)
plt.imshow(sobel, cmap='gray')
plt.title("Sobel Edge")
plt.axis('off')

plt.subplot(1,3,3)
plt.imshow(canny, cmap='gray')
plt.title("Canny Edge")
plt.axis('off')

plt.tight_layout()
plt.show()
```



Task 2: Object Representation

- Detect contours
- Draw bounding boxes around detected objects
- Compute object area and perimeter

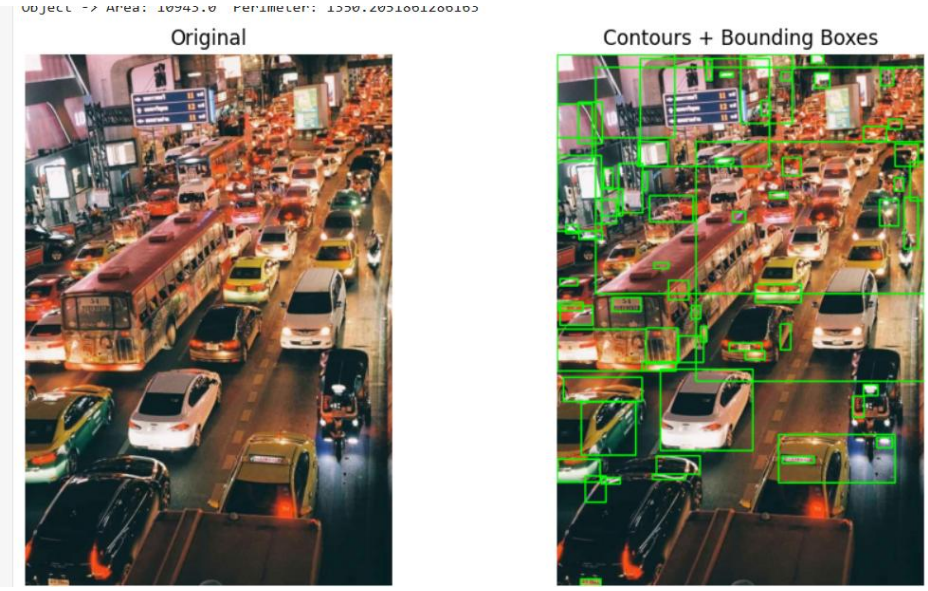
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Read image
img = cv2.imread("img.jpg")

if img is None:
    print("Error: Image not found")
else:
    # Convert to RGB
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Step 2: Convert to grayscale
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Step 3: Threshold (to detect objects clearly)
    _, thresh = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

    # Step 4: Find contours
    contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

    # Copy image for drawing
    output = img_rgb.copy()

    # Step 5: Loop through contours
    for cnt in contours:
        # Ignore very small noise
        if cv2.contourArea(cnt) > 100:
            # Bounding box
            x, y, w, h = cv2.boundingRect(cnt)
            cv2.rectangle(output, (x, y), (x+w, y+h), (0,255,0), 2)

            # Area and perimeter
            area = cv2.contourArea(cnt)
            perimeter = cv2.arcLength(cnt, True)

            print("Object -> Area:", area, " Perimeter:", perimeter)

    # ----------------------------
    # Display
    # ----------------------------
    plt.figure(figsize=(10,5))

    plt.subplot(1,2,1)
    plt.imshow(img_rgb)
    plt.title("Original")
    plt.axis('off')
```

```
plt.subplot(1,2,2)
plt.imshow(output)
plt.title("Contours + Bounding Boxes")
plt.axis('off')

plt.tight_layout()
plt.show()
```

output

```
Object -> Area: 250.5  Perimeter: 133.35533845424652
Object -> Area: 132.5  Perimeter: 90.18376541137695
Object -> Area: 102.0  Perimeter: 120.16652023792267
Object -> Area: 128.0  Perimeter: 127.11269783973694
Object -> Area: 1369.5  Perimeter: 249.09545278549194
Object -> Area: 382.0  Perimeter: 142.76955199241638
Object -> Area: 306.0  Perimeter: 237.82337498664856
Object -> Area: 237.0  Perimeter: 88.42640602588654
Object -> Area: 3202.0  Perimeter: 1037.2346230745316
Object -> Area: 1185.0  Perimeter: 532.2985519170761
Object -> Area: 116.0  Perimeter: 110.28427076339722
Object -> Area: 251.0  Perimeter: 78.42640590667725
Object -> Area: 1424.0  Perimeter: 601.1025931835175
Object -> Area: 10097.5  Perimeter: 973.3940042257309
Object -> Area: 518.5  Perimeter: 146.5269113779068
Object -> Area: 297.0  Perimeter: 116.76955187320709
Object -> Area: 292.0  Perimeter: 152.9116872549057
Object -> Area: 514.5  Perimeter: 178.26702547073364
Object -> Area: 5996.0  Perimeter: 908.3818110227585
Object -> Area: 818.0  Perimeter: 499.6467492580414
Object -> Area: 148.5  Perimeter: 71.21320307254791
Object -> Area: 178.0  Perimeter: 108.56854128837585
Object -> Area: 116.0  Perimeter: 78.08326005935669
Object -> Area: 599.5  Perimeter: 266.5512965917587
Object -> Area: 160.0  Perimeter: 140.42640614509583
Object -> Area: 258.5  Perimeter: 169.63960909843445
Object -> Area: 1295.5  Perimeter: 328.2081503868103
Object -> Area: 386.0  Perimeter: 157.3380926847458
Object -> Area: 112.0  Perimeter: 73.11269783973694
Object -> Area: 106.0  Perimeter: 63.455843687057495
Object -> Area: 248.0  Perimeter: 136.56854152679443
Object -> Area: 116.0  Perimeter: 63.11269795894623
Object -> Area: 112.5  Perimeter: 79.84061932563782
Object -> Area: 491.5  Perimeter: 191.9238796234131
Object -> Area: 453.0  Perimeter: 168.3675309419632
Object -> Area: 918.5  Perimeter: 285.0365778207779
Object -> Area: 739.5  Perimeter: 425.2619730234146
Object -> Area: 161.5  Perimeter: 84.04163014888763
Object -> Area: 532.0  Perimeter: 213.82337486743927
Object -> Area: 128.0  Perimeter: 109.25483310222626
Object -> Area: 158.5  Perimeter: 75.89949476718903
```

Object -> Area: 224.0 Perimeter: 91.94112491607666
Object -> Area: 1141.0 Perimeter: 305.62236404418945
Object -> Area: 867.0 Perimeter: 208.1665198802948
Object -> Area: 161.0 Perimeter: 68.97056245803833
Object -> Area: 205.5 Perimeter: 172.26702558994293
Object -> Area: 4640.0 Perimeter: 751.8132693767548
Object -> Area: 369.0 Perimeter: 179.0538226366043
Object -> Area: 60535.5 Perimeter: 6068.939703345299
Object -> Area: 1730.5 Perimeter: 173.01219260692596
Object -> Area: 383.5 Perimeter: 241.52185833454132
Object -> Area: 113.0 Perimeter: 85.25483310222626
Object -> Area: 4030.5 Perimeter: 309.92387986183167
Object -> Area: 148.5 Perimeter: 178.89444279670715
Object -> Area: 136.0 Perimeter: 110.56854140758514
Object -> Area: 368.5 Perimeter: 126.18376553058624
Object -> Area: 113.5 Perimeter: 54.72792184352875
Object -> Area: 109.5 Perimeter: 93.01219248771667
Object -> Area: 197.0 Perimeter: 160.71067678928375
Object -> Area: 24102.5 Perimeter: 5335.292953372002
Object -> Area: 186.0 Perimeter: 85.4558436870575
Object -> Area: 18301.0 Perimeter: 1461.0579977035522
Object -> Area: 241.5 Perimeter: 63.55634891986847
Object -> Area: 3578.0 Perimeter: 769.043717622757
Object -> Area: 10943.0 Perimeter: 1350.2051861286163



Task 3: Feature Extraction

- Apply ORB / SIFT / SURF
- Visualize keypoints and descriptors

```
import cv2
import matplotlib.pyplot as plt
```

```
# Step 1: Read image
img = cv2.imread("img.jpg")

if img is None:
    print("Error: Image not found")
else:
    # Convert to RGB
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Convert to grayscale
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # ------------------------------
    # Step 2: ORB Feature Detector
    # ------------------------------
    orb = cv2.ORB_create(nfeatures=500)

    # Detect keypoints and descriptors
    keypoints, descriptors = orb.detectAndCompute(gray, None)

    print("Number of Keypoints:", len(keypoints))
    print("Descriptor Shape:", descriptors.shape)

    # ------------------------------
    # Step 3: Draw Keypoints
    # ------------------------------
    img_kp = cv2.drawKeypoints(
        img_rgb, keypoints, None,
        color=(0,255,0),
        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
    )

    # ------------------------------
    # Display
    # ------------------------------
    plt.figure(figsize=(8,5))
    plt.imshow(img_kp)
    plt.title("ORB Feature Keypoints")
    plt.axis('off')
    plt.show()
```

Task 4: Comparative Analysis

- Compare edge detectors and feature extractors
- Explain how features help in traffic monitoring

Feature	Sobel	Canny
Accuracy	Low	High
Noise Handling	Poor	Good
Edge Thickness	Thick	Thin
Complexity	Simple	Advanced

Feature	ORB	SIFT	SURF
Speed	Fast	Slow	Faster than SIFT
Accuracy	Moderate	High	High
License	Free	Restricted (earlier)	Restricted
Use Case	Real-time	High accuracy tasks	Balanced